

Lab 4: MIPS Programming

AIM: This lab will give you practice running and debugging assembly programs using the MARS simulator.

After this lab you will be able to:

- Load MIPS programs (assembly language) and execute them
- Examine memory locations
- Examine registers
- Execute programs step by step
- How to write assembly prog

Execute the sample codes of MIPS as an exercise and record the outputs:

a. Code 1:

```
main: addi $t1, $zero, 97
# sample1.asm
```

Examine the content of the register

Code 11:

```
.text
main:
    addi $t0, $zero, 25
    addi $t2, $t0, 15
```

Your Answer: \$0 = _____ \$t1 = _____

Add your own experiments with different code segments.

Array Declaration with Initialization

```
.data
data1: .byte 1,2,3,4,5,6
vowels: .byte 'a', 'e', 'i', 'o', 'u'
pow2: .word 1, 2, 4, 8, 16, 32, 64, 128
```

Examine the content of the memory (data segment) starting at **0x 1001 0000**

Record Your Answer:

For example:

Address	Value (decimal)
0x10010000	1
0x10010001	2
0x10010002	3
0x10010003	4
0x10010004	5
0x10010005	6

b. Code 2:

```
.data
msg: .asciiz "Hello"
.text
main: li $v0, 4
      la $a0, msg
      syscall
      # sample2.asm
```

Your Answer:

Figure out where and how is the string “Hello” stored? Write out the ASCII values, in hexadecimal form, of the characters ‘H’, ‘e’, ‘l’ and ‘o’ below:

‘H’: _____ ‘e’: _____ ‘l’: _____ ‘o’: _____

c. Code 3:

```
# Data stored in memory
.data
value:                                # a value stored in memory
.word 7
result:                               # where result is stored
.word 0

# The program
.text
lw    $s0, value
move  $s1, $s0
```

```

sll    $s0, $s0, 2
add    $s0, $s0, $s1
sw     $s0, result

```

d. Code 4:

```

li     $v0, 16      # system call for close file
move   $a0, $s6     # file descriptor to close
syscall                    # close file

```

e. Code 5

```

.data
msg1: .asciiz "Enter the first number: "
msg2: .asciiz "\nEnter the second number: "
result: .asciiz "\nThe result of addition is: "

.text
li $v0, 4
la $a0, msg1
syscall

li $v0, 5
syscall
move $t1, $v0

li $v0, 4
la $a0, msg2
syscall

li $v0, 5
syscall
move $t2, $v0

Add $t3, $t1, $t2

li $v0, 4
la $a0, msg3
syscall

li $v0, 1
move $a0, $t3
syscall

li $v0, 10
syscall

```

Show the usage of each system call using codes of your own. The usage of system call codes 4 and 10 are shown in the sample code below.

MIPS provides a small set of operating-system-like services through the system call (syscall) instruction. To request a service, a program loads the system call code (see Table 1) into register \$v0 and the arguments into registers \$a0 . . . \$a3 (or \$f12 for floating point values). System calls that return values load their result in register \$v0 (or \$f0 for floating point results).

Table: System services.

Service	System Call Code	Arguments	Result
print_int	1	\$a0 = integer	
print_float	2	\$f12 = float	
print_double	3	\$f12 = double	
print_string	4	\$a0 = string	
read_int	5		integer (in \$v0)
read_float	6		float (in \$f0)
read_double	7		double (in \$f0)
read_string	8	\$a0 = buffer, \$a1 = length	
sbrk	9	\$a0 = amount	address (in \$v0)
exit	10		
print_character	11	\$a0 = character	
read_character	12		character (in \$v0)
open	13	\$a0 = filename,	file descriptor (in \$v0)
		\$a1 = flags, \$a2 = mode	
read	14	\$a0 = file descriptor,	bytes read (in \$v0)
		\$a1 = buffer, \$a2 = count	
write	15	\$a0 = file descriptor,	bytes written (in \$v0)
		\$a1 = buffer, \$a2 = count	
close	16	\$a0 = file descriptor	0 (in \$v0)
exit2	17	\$a0 = value	

For example, to print two strings, use the following command:

```

.text
main:
la $a0, str1
la $t0, str2

li $v0, 4
syscall

move $a0, $t0
li $v0, 4
syscall

li $v0, 10
syscall

.data
str1: .asciiz "I am in IIT Patna"
str2: .asciiz "I am doing my lab work"

```

For each of the system call code, create your own example and test the system call functionality

Points to Remember:

Directives (instructions to assembler, not MIPS instructions)

```

.text      # following instructions placed in text
.data      # following objects placed in data

.globl     # make symbol available globally

a: .space 18  # uchar a[18]; or uint a[4];
   .align 2   # align next object on 22-byte addr

i: .word 2    # unsigned int i = 2;
v: .word 1,3,5 # unsigned int v[3] = {1,3,5};
h: .half 2,4,6 # unsigned short h[3] = {2,4,6};
b: .byte 1,2,3 # unsigned char b[3] = {1,2,3};
f: .float 3.14 # float f = 3.14;

s: .asciiz "abc"
   # char s[4] {'a','b','c','\0'};
t: .ascii "abc"
   # char s[3] {'a','b','c'};

```

Example:

C	Simplified C	MIPS Assembler
-----	-----	-----
int x = 5;	int x = 5;	x: .word 5
int y = 3;	int y = 3;	y: .word 3
int z;	int z;	z: .space 4
	int t;	...
		lw \$t0, x
		lw \$t1, y
z = 5*(x+y);	t = x+y;	add \$t0, \$t0, \$t1
		li \$t1, 5
	t = 5*t;	mul \$t0, \$t0, \$t1
	z = t;	sw \$t0, z

Task:

```
.data
var1: .byte 3, -2, 'A'
var2: .half 1, 256, 0xffff
var3: .word 0x3de1c74, 0xff
.align 3
str1: .asciiz "CS2206 Architecture Course"
```

Show the content of each byte of the allocated memory, **in hexadecimal** for the above data definitions. The **Little Endian** byte ordering is used to order the bytes within words and halfwords. Fill the symbol table showing **all labels** and their **starting address**. The ASCII code of character 'A' is 0x41, and '0' is 0x30. Indicate which bytes are skipped or unused in the data segment.

Address	Byte 0	Byte 1	Byte 2	Byte 3		Label	Address
0x10010000						var1	0x10010000
....	...	...	...	...		var2	
						var3	
0x100100XX						str1	

Submission:

Write basic code in the lab record along with outputs and demonstrate.